Augmenting LLM Code Translation with Compiler Analysis for C to Triton Kernel Generation

Xiao Qin
University of Leeds
Leeds, United Kingdom
ppnh4382@leeds.ac.uk

Chunwei Xia
University of Leeds
Leeds, United Kingdom

Zheng Wang
University of Leeds
Leeds, United Kingdom

ABSTRACT

Emerging programming models like Triton enable developers to better exploit modern accelerators, but translating legacy code to Triton remains challenging. While Large Language Models (LLMs) show promise for code translation, they often generate incorrect or suboptimal implementations due to a lack of precise parallelization reasoning. We present TritonPilot, a compiler-assisted LLM framework for translating C loop nests to Triton kernels. TritonPilot extracts parallelization properties using compiler-based analysis and encodes them as structured analysis facts in the LLM prompt, enabling correct and performance-aware parallel code synthesis. A profiling feedback loop further improves performance by feeding GPU hardware utilization metrics back to the LLM for iterative optimization. On PolyBench/C, TritonPilot achieves 97% correctness, outperforming both an autonomous LLM agent with tool use (80%) and an unguided LLM baseline (93%). At 8× problem sizes, the generated Triton kernels achieve a median 24.6× speedup, compared to 20.7× for the agent and 10.8× for the unguided baseline. Profiling feedback improves 9 of 29 kernels at standard sizes and 12 of 21 at larger sizes. TritonPilot also passes 95% of 151 TSVC kernels and generalizes to 6 of 8 Rodinia/ECP application kernels. These results show that compiler analysis outperforms autonomous LLM agents for kernel generation correctness and performance.

CCS CONCEPTS

• **Software and its engineering** → **Compilers**; • **Computing methodologies** → *Parallel computing methodologies*.

KEYWORDS

LLM code generation, GPU kernels, Triton, polyhedral analysis

ACM Reference Format:
Xiao Qin, Chunwei Xia, and Zheng Wang. 2026. Augmenting LLM Code Translation with Compiler Analysis for C to Triton Kernel Generation. In *Proceedings of Workshop on AI for High Performance Compiler Construction (AI4HPCC '26).* ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

AI4HPCC '26, July 6–9, 2026, Belfast, Northern Ireland, UK
2026. ACM ISBN 978-x-xxxx-xxxx-x/YY/MM. . . $15.00
https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 INTRODUCTION

Accelerators such as GPUs are now essential for high-performance scientific computing, yet much existing software is still written in legacy C code that is not designed for parallel execution on modern accelerators. Translating these legacy programs to accelerator-friendly implementations is challenging, as it requires identifying parallelism, restructuring computation, and managing synchronization, tasks that require expertise in programming models and hardware.

Emerging programming models such as Triton [1] simplify GPU programming by providing higher-level abstractions for writing efficient kernels. However, mapping legacy C code to Triton still requires careful reasoning about loop-level parallelism, data dependencies, and execution order. Developers must determine which loop dimensions can be safely parallelized, how memory accesses should be organized, and when synchronization is required - decisions that are subtle and error-prone in practice.

Large Language Models (LLMs) have recently shown promise in code translation, including generating GPU kernels from high-level descriptions [2, 3]. However, LLMs lack precise reasoning about program semantics, especially data dependencies. As a result, they often produce incorrect or inefficient code, including unsafe parallelization that leads to data races, suboptimal kernel structures, and missing synchronization.

In this paper, we present TritonPilot, a system that augments LLM-based translation with compiler analysis. TritonPilot extracts parallelization properties from C loop nests using polyhedral analysis (PET) [4] and LLVM dependence analysis [5], and encodes them as structured guidance in the LLM prompt. This allows the model to generate correct and performance-aware Triton code without requiring it to perform dependence analysis itself.

We evaluate TritonPilot on PolyBench/C, TSVC, and application kernels on an NVIDIA RTX 3090 GPU. TritonPilot achieves 97% correctness on PolyBench/C and 95% on TSVC, improving over unguided baselines. At 8× problem sizes, the generated kernels achieve a median 24.6× speedup. We further demonstrate generalization on 8 Rodinia and ECP application kernels. This paper makes the following contributions:

- A compiler-assisted LLM framework for translating C loop nests to Triton, with a unified analysis module that derives all parallelization constraints from a single polyhedral analysis invocation and encodes them as structured facts.

- A profiling-guided optimization loop that feeds GPU hardware utilization metrics (from NVIDIA Nsight Compute) back to the LLM, enabling iterative performance improvement while preserving correctness.
- Empirical evaluation on 189 kernels across PolyBench/C, TSVC, Rodinia, and ECP benchmarks, showing that compiler analysis improves correctness and profiling feedback significantly improves performance.

2 OUR APPROACH

Figure 1 shows the TritonPilot workflow. Given a C source file, TritonPilot extracts loop nests, analyzes their parallelization properties, constructs a structured prompt, invokes an LLM to generate Triton code, and refines performance using GPU profiling feedback. TritonPilot is implemented in approximately 14K lines of Python, using PET/isl [8] for polyhedral analysis and LLVM 17 [5] for fallback dependence analysis.

2.1 Unified Analysis

A key design goal of TritonPilot is to derive all parallelization constraints from a *single* invocation of the polyhedral extraction tool, avoiding redundant analysis passes. TritonPilot calls PET [4] once per kernel to extract a polyhedral representation consisting of iteration domains, access relations, and schedules as ISL [8] objects. From this single representation, it derives four categories of constraints:

(i) Parallelization safety. For each loop dimension, TritonPilot checks for cross-iteration dependences by composing ISL access relations: given read map R and write map W to the same array, the conflict relation $R \circ W^{-1}$ is intersected with the iteration domain and the identity map is subtracted. A non-empty result marks the dimension as unsafe. For example, in `gemm`, the analysis confirms that both `i` and `j` are safe since each `C[i][j]` is updated independently.

(ii) WAR dependences. Write-after-read conflicts are identified when an array is both read and written at different iteration points. The analysis distinguishes RAW (sequential dependence) from WAR (parallel hazard) and reports which loops carry the conflict.

(iii) Memory access patterns. Stencil patterns (neighboring-element reads requiring synchronization barriers), triangular access (bounds depending on outer variables, e.g., `j < i`), and cross-phase dependencies (phases writing different arrays under the same outer loop) are detected from the access structure.

(iv) Reductions. Scalar writes combined with array reads indicate reduction patterns, mapped to Triton primitives (`tl.sum()`, `tl.dot()`).

When polyhedral analysis is not applicable (e.g., due to non-affine subscripts or pointer-based access), TritonPilot falls back to LLVM's `DependenceAnalysis` pass, parsing direction vectors to conservatively determine which loop levels carry dependences.

Example: `jacobi_1d`. Consider the Jacobi 1D stencil, which iterates `TSTEPS` times over arrays `A` and `B`:

```
for (t = 0; t < TSTEPS; t++) {
  for (i = 1; i < N-1; i++)
    B[i] = 0.33333 * (A[i-1] + A[i] + A[i+1]);
  for (i = 1; i < N-1; i++)
    A[i] = 0.33333 * (B[i-1] + B[i] + B[i+1]);
}
```

PET extracts the domains {`S_2[t,i]`} and {`S_3[t,i]`} with accesses to `A` and `B` at offsets `i-1`, `i`, `i+1`. The unified analysis derives: (1) `i` is safe to parallelize (no cross-iteration conflict on `i`); (2) `t` is unsafe (cross-phase: phase 1 writes `B`, phase 2 reads `B`); (3) stencil pattern detected (neighboring-element reads); (4) WAR on `B` between phases. The prompt states these facts; the LLM generates a single-CTA kernel with `tl.debug_barrier()` between phases.

2.2 Prompt Construction

All analysis results are stored in a unified JSON schema with fixed fields, regardless of which patterns the kernel exhibits. A single, pattern-agnostic function renders the analysis into structured natural-language text. This design principle—*tell the LLM what is true about the kernel, rather than prescribing what to do*—ensures that new analysis passes can be added by extending the schema without modifying prompt templates. The prompt includes:

- The original C source code and the loop body to translate.
- Analysis facts: valid/invalid parallelization dimensions with reasons, WAR dependence warnings, detected memory access patterns, and reduction types.
- The required Triton function signature and coding rules (e.g., `tl.constexpr` for dimensions, vectorized access patterns, avoiding unsupported Python constructs in `@triton.jit` kernels).

The LLM receives the analysis as *facts* ("dimension `j` is safe to parallelize; dimension `t` has cross-phase dependencies; stencil pattern detected") and autonomously decides the implementation strategy, rather than following a prescriptive code template. This approach reduces prompt maintenance burden and allows the LLM to apply its own knowledge of Triton programming patterns.

2.3 Generation and Iterative Refinement

TritonPilot invokes the LLM (Claude Sonnet 4 [7]) and validates the generated kernel against the C reference (compiled with GCC or Clang) using identical random inputs and element-wise FP32 tolerance checks. On validation failure, TritonPilot classifies the error into one of four categories: *(i)* compilation or runtime errors (syntax errors, missing imports, Triton JIT compilation failures); *(ii)* numerical errors (output values exceed tolerance thresholds); *(iii)* missing synchronization barriers (detected via pre-validation for stencil kernels where `tl.debug_barrier()` is required between phases); *(iv)* low performance (speedup $<0.1\times$, indicating the kernel is effectively sequential). Each category triggers a targeted retry prompt with error-specific feedback and the previous attempt's code. This process repeats for up to 10

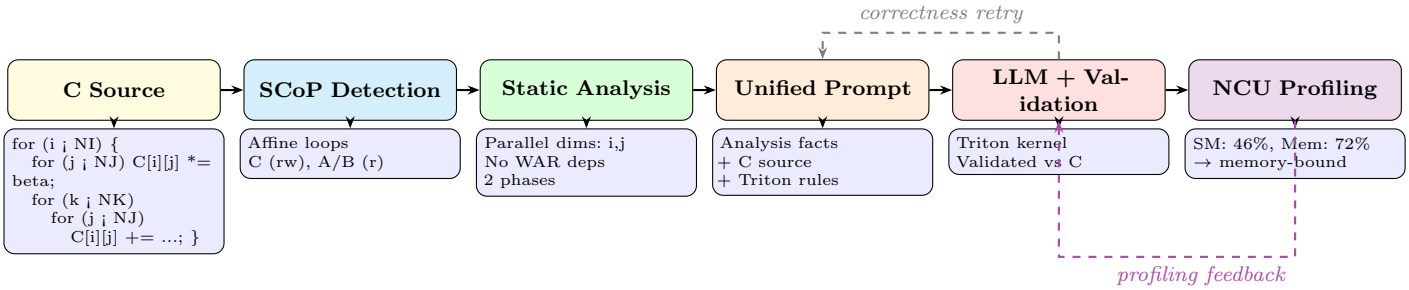

Figure 1: The TritonPilot workflow. Dashed arrows show iterative loops: the gray loop retries on correctness failures; the violet loop feeds NCU profiling metrics back to the LLM for performance optimization.

attempts, with a context reset after the 6th attempt to avoid error accumulation in the conversation history.

2.4 Profiling-Guided Optimization

After a kernel passes correctness validation, TritonPilot enters a profiling-guided optimization loop. NVIDIA Nsight Compute (NCU) profiles the kernel, collecting SM (compute) throughput as a percentage of peak, memory throughput as a percentage of peak, warp occupancy, and L1/L2 cache hit rates. From these metrics, TritonPilot classifies the performance bottleneck: *Compute-bound* (SM throughput dominant): guided toward `tl.dot()` and reduced redundant computation. *Memory-bound* (memory throughput dominant): guided toward tiling and coalesced access. *Latency-bound* (both <20%): guided toward increased grid size and reduced sequential work.

The bottleneck classification, raw metrics, and current code are fed back to the LLM. The optimized kernel is re-validated and re-benchmarked; it replaces the current best only if both correct and faster. This loop runs up to 3 iterations, with NCU results cached when the implementation is unchanged.

3 EVALUATION SETUP

Platforms. All experiments are conducted on a GPU workstation equipped with an NVIDIA RTX 3090 GPU (24 GB memory, 82 SMs, 936 GB/s memory bandwidth) and a dual-socket Intel Xeon Gold 5218R CPU (80 logical cores, ∼140 GB/s memory bandwidth). We use PyTorch 2.5.1, Triton 3.1.0, CUDA 12.4, GCC 11.4.0, and Claude Sonnet 4 [7] as the underlying LLM for all configurations.

Benchmarks. We evaluate on three benchmark suites: (1) *PolyBench/C* [6]: 30 numerical kernels (stencils, linear algebra, data mining) at default sizes (N=60–120) and 8× scale (N=480–960); (2) *TSVC* [15]: 151 loop kernels covering linear traversals, reductions, indirect addressing, and loop-carried dependences; (3) 8 application kernels from Rodinia [13] (SRAD, lud, gaussian, lavaMD, hotspot, pathfinder) and ECP proxy applications [14] (miniMD LJ force, miniFE SpMV).

Configurations. On PolyBench/C (our primary benchmark), we compare three configurations to isolate the contribution of each component:

- *Agent*: An autonomous LLM agent using Claude's tool-use API. The agent can write Triton code, run correctness tests, and run benchmarks, seeing the full raw output at each step. It decides its own retry strategy with no external guidance. No compiler analysis is provided. Up to 10 correctness turns + 3 optimization turns.
- *NA* (No Analysis): The pipeline-managed baseline. The LLM receives only the C source code. On failure, it receives *classified* error feedback (e.g., "numerical error, max_error=0.01") and the previous attempt. Same turn budget.
- *WA+Prof* (With Analysis + Profiling): Our full method. The LLM receives compiler-derived analysis facts (parallelization safety, WAR dependences, memory patterns, reductions). After correctness, NCU profiling metrics guide up to 3 optimization iterations.

All three configurations use the same LLM (Claude Sonnet 4), the same test infrastructure, and the same C reference implementations, ensuring a fair comparison. On TSVC and application kernels, we report WA+Prof results only.

Evaluation methodology. Speedups are measured over 50 iterations after 5 warmup iterations. Each iteration includes array re-initialization (GPU `clone()` for Triton, CPU `numpy.copy()` from pre-allocated source arrays for the C reference) to ensure in-place kernels start from identical data. Triton timing includes `cuda.synchronize()`. Correctness is determined by element-wise comparison with the C reference using per-kernel FP32 tolerance thresholds (default: absolute $< 10^{-3}$, relative $< 10^{-4}$; relaxed for kernels with long recurrence chains, e.g., `lu`: absolute < 5.0).

4 EXPERIMENTAL RESULTS

4.1 PolyBench/C (30 Kernels)

Tables 1 and 2 compare three configurations on PolyBench/C at two problem scales.

Table 1: PolyBench/C results at 1× sizes. Agent = autonomous LLM with tool use (no analysis); NA = LLM with classified error feedback (no analysis); WA+Prof = our method (compiler analysis + NCU profiling feedback).

Metric	Agent	NA	WA+Prof
Pass rate	24/30 (80%)	28/30 (93%)	29/30 (97%)
First-try passes	11	8	12
Median speedup	0.60×	0.88×	0.76×
Mean speedup	0.87×	1.64×	1.24×
Kernels > 1×	7/23	12/28	11/29

Table 2: PolyBench/C results at 8× sizes (N: 60→480, matrices: 120→960). Speedups vs. single-threaded C.

Metric	Agent	NA	WA+Prof
Pass rate	17/30 (57%)	26/30 (87%)	27/30 (90%)
Median speedup	20.7×	10.8×	24.6×
Kernels > 1×	13/15	14/20	18/21

Correctness. Our method (WA+Prof) achieves the highest correctness at both scales: 97% at 1× and 90% at 8×. The autonomous agent, despite having full tool access, achieves only 80% at 1× and 57% at 8×—the weakest of all three. This is notable: the agent can autonomously test, debug, and iterate, yet compiler analysis produces more correct kernels with a simpler pipeline. The agent wastes turns on ineffective fixes without understanding root causes (e.g., repeatedly adjusting unrelated code when the real issue is parallelizing a dimension with loop-carried dependences). The NA baseline (93%) outperforms the agent, suggesting that classified error feedback is more effective than autonomous debugging when the LLM lacks domain-specific analysis.

Performance. At 1× sizes, median speedups are modest across all configurations due to GPU launch overhead at small N. At 8× sizes, WA+Prof achieves 24.6× median, compared to 10.8× for NA and 20.7× for the agent, because compiler analysis identifies the correct parallelization strategy upfront. Top kernels: `3mm` (2041×), `2mm` (2004×), `gemm` (451×).

4.2 Profiling-Guided Optimization

Profiling feedback improved 9 of 29 kernels at 1× and 12 of 21 benchmarked kernels at 8× (Table 3). The improvements are most pronounced on latency-bound kernels where NCU reveals low SM and memory utilization (<20%), indicating insufficient parallelism. For example, `3mm` improves from 0.10× to 4.01× at 1× after the LLM transforms a sequential loop into a multi-CTA grid in response to the latency-bound diagnosis. At 8×, `gemm` improves from 120× to 451× after switching to tiled matrix multiplication. Correctness is preserved: each optimized kernel is re-validated, and the majority of optimization attempts that break correctness are automatically rejected.

Table 3: Representative profiling feedback improvements. "Before" = first correct kernel; "After" = best after NCU-guided iterations.

Kernel	Before	After	Scale
3mm	0.10×	4.01×	1×
covariance	0.11×	3.66×	1×
2mm	1.48×	2.97×	1×
syrk	1.65×	2.07×	1×
gemm	120×	451×	8×
correlation	12×	51×	8×
atax	0.4×	15×	8×
syrk	24×	62×	8×

Table 4: Application kernels: Triton speedup vs. single-threaded C.

Source	Kernel	WA/ST	Pass?
ECP	LJ force	24.1×	Y
ECP	SpMV	7.1×	Y
Rodinia	SRAD	12.8×	Y
Rodinia	pathfinder	3.6×	Y
Rodinia	lud	1.3×	Y
Rodinia	gaussian	0.6×	Y
Rodinia	hotspot	—	N
Rodinia	lavaMD	—	N

4.3 TSVC (151 Kernels)

We apply TritonPilot to 151 TSVC [15] kernels covering linear traversals, reductions, indirect addressing, and loop-carried dependences. TritonPilot passes 143/151 kernels (95%), with 91 passing on the first attempt. The mean speedup over single-threaded C is 2.78×, with a maximum of 267×. The 8 failing kernels involve complex patterns such as linked-list traversals, computed gotos, and deeply nested conditional recurrences that fall outside the scope of polyhedral analysis. Despite using a general-purpose analysis module rather than pattern-specific templates, TritonPilot achieves comparable pass rates to prior work that relied on hand-crafted prompt engineering for each kernel pattern.

4.4 Application Kernels

To evaluate generalization beyond loop benchmarks, we apply TritonPilot to 3 Rodinia [13] and 5 ECP/Rodinia application kernels (Table 4). 6 of 8 pass correctness with analysis guidance. The highest speedups come from compute-intensive kernels with regular parallelism: `LJ_force` (24.1×, Lennard-Jones force computation with neighbor-list traversal) and `SRAD` (12.8×, multi-phase stencil). Two kernels fail: `hotspot` (complex 2D stencil with boundary conditions requiring careful masking) and `lavaMD` (irregular particle interactions with

variable-length neighbor lists), both requiring domain-specific optimizations beyond what the unified analysis provides.

Limitations. LLM nondeterminism affects results: the same kernel may pass or fail across runs. The profiling feedback loop is limited by the LLM's ability to restructure code without breaking correctness, motivating future integration with search-based or RL-based methods.

5 RELATED WORK

LLM Code Generation for GPUs. Chen et al. [2] find frequent correctness issues in ChatGPT-generated CUDA kernels. KernelBench [3] reports 20–40% functional correctness for LLM-generated GPU kernels without guidance. Recent works improve generation through RL fine-tuning: Dr. Kernel [16] uses profiling-based rewards, Tehrani et al. [17] RL-fine-tune GPT-5, and DICE [18] applies RL to diffusion LLMs. Search-based methods keep the LLM frozen: AVO [19] uses LLM agents as evolutionary operators, K-Search [20] decouples planning from code instantiation, and OptiML [21] uses MCTS with profiler feedback. KernelBlaster [22] enables cross-task learning via persistent memory. For non-GPU targets, AscendKernelGen [23] and AscendCraft [24] address NPU kernels. These works focus on *how* to generate/optimize kernels; our work is orthogonal, addressing *what* parallelism is safe via compiler analysis. The two directions are complementary: compiler-derived constraints could serve as correctness rewards in RL frameworks or safety priors in search.

Polyhedral Compilation and Auto-Scheduling. PPCG [12] generates CUDA from affine loop nests but produces low-level code that lacks Triton's block-level optimizations. TVM [9] and Halide [10] use auto-scheduling with learned cost models. TritonPilot is complementary: it uses polyhedral analysis to extract *what* parallelism is safe, then delegates the *how* to the LLM, which can handle non-affine constructs, irregular access patterns, and Triton abstractions that polyhedral compilers cannot target directly.

6 CONCLUSION

We presented TritonPilot, a system that combines compiler-based analysis with LLM code generation for translating C loop nests to Triton GPU kernels. A unified analysis module calls PET once per kernel and derives all parallelization constraints (safe dimensions, WAR dependences, reductions, memory patterns) from the single polyhedral representation. These are rendered as structured facts in the LLM prompt via a pattern-agnostic renderer, replacing pattern-specific templates.

TritonPilot achieves 97% correctness on PolyBench/C (29/30) and 95% on TSVC (143/151), outperforming both an autonomous LLM agent with tool use (80% on PolyBench) and an unguided LLM baseline (93%). At 8× sizes, the correctness gap widens further: 90% (WA) vs. 57% (agent) vs. 87% (NA). This demonstrates that compiler analysis provides stronger correctness guarantees than autonomous tool use.

Profiling feedback improved 9 of 29 kernels at 1× and 12 of 21 at 8×. Future work will integrate compiler analysis and profiling metrics as reward signals in search-based or RL-based optimization frameworks.

REFERENCES

[1] P. Tillet, H. T. Kung, and D. Cox. Triton: An intermediate language and compiler for tiled neural network computations. In *Proc. MAPL*, 2019.

[2] Y. Chen, S. Huang, D. Zheng, et al. ChatGPT for CUDA programming: Evaluating LLM-generated GPU code. *arXiv preprint arXiv:2404.12000*, 2024.

[3] A. Ouyang, S. Zheng, et al. KernelBench: Can LLMs write efficient GPU kernels? *arXiv preprint arXiv:2404.12489*, 2024.

[4] S. Verdoolaege and T. Grosser. Polyhedral extraction tool. In *Proc. IMPACT*, 2012.

[5] C. Lattner and V. Adve. LLVM: A compilation framework for lifelong program analysis & transformation. In *Proc. CGO*, 2004.

[6] L.-N. Pouchet. Polybench/C: The polyhedral benchmark suite, version 4.2.1. https://web.cs.ucla.edu/~pouchet/software/polybench/, 2016.

[7] Anthropic. Claude Sonnet 4 model card. https://www.anthropic.com/claude, 2025.

[8] S. Verdoolaege. isl: An integer set library for the polyhedral model. In *Proc. ICMS*, 2010.

[9] T. Chen, T. Moreau, Z. Jiang, et al. TVM: An automated end-to-end optimizing compiler for deep learning. In *Proc. OSDI*, 2018.

[10] J. Ragan-Kelley, C. Barnes, A. Adams, et al. Halide: A language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. In *Proc. PLDI*, 2013.

[11] U. Bondhugula, A. Hartono, J. Ramanujam, and P. Sadayappan. A practical automatic polyhedral parallelizer and locality optimizer. In *Proc. PLDI*, 2008.

[12] S. Verdoolaege, J. Carlos Juega, A. Cohen, et al. Polyhedral parallel code generation for CUDA. *ACM TACO*, 9(4), 2013.

[13] S. Che, M. Boyer, J. Meng, et al. Rodinia: A benchmark suite for heterogeneous computing. In *Proc. IISWC*, 2009.

[14] M. A. Heroux, D. W. Doerfler, P. S. Crozier, et al. Improving performance via mini-applications. Sandia National Laboratories Technical Report SAND2009-5574, 2009.

[15] D. Callahan, J. Dongarra, and D. Levine. Vectorizing compilers: A test suite and results. In *Proc. Supercomputing*, 1988.

[16] W. Liu, J. Xu, Y. Li, et al. Dr. Kernel: Reinforcement learning done right for Triton kernel generations. *arXiv preprint arXiv:2602.05885*, 2026.

[17] A. Tehrani, Y. Emara, E. Wissam, et al. Fine-tuning GPT-5 for GPU kernel generation. *arXiv preprint arXiv:2602.11000*, 2026.

[18] H. Bai, L. Kong, X. Chen, et al. DICE: Diffusion large language models excel at generating CUDA kernels. *arXiv preprint arXiv:2602.11715*, 2026.

[19] T. Chen, Z. Ye, B. Xu, et al. AVO: Agentic variation operators for autonomous evolutionary search. *arXiv preprint arXiv:2603.24517*, 2026.

[20] S. Cao, Z. Mao, J. E. Gonzalez, and I. Stoica. K-Search: LLM kernel generation via co-evolving intrinsic world model. *arXiv preprint arXiv:2602.19128*, 2026.

[21] A. Bhattacharjee, H. Ping, S. V. Le, et al. OptiML: An end-to-end framework for program synthesis and CUDA kernel optimization. *arXiv preprint arXiv:2602.12305*, 2026.

[22] K. S. Dong, S. Modi, D. Nikiforov, et al. KernelBlaster: Continual cross-task CUDA optimization via memory-augmented in-context reinforcement learning. *arXiv preprint arXiv:2602.14293*, 2026.

[23] X. Cao, J. Zhai, P. Li, et al. AscendKernelGen: A systematic study of LLM-based kernel generation for neural processing units. *arXiv preprint arXiv:2601.07160*, 2026.

[24] Z. Wen, S. Shao, Z. Li, et al. AscendCraft: Automatic Ascend NPU kernel generation via DSL-guided transcompilation. *arXiv preprint arXiv:2601.22760*, 2026.